MILVUS ENGINEERING PROTOTYPE · 31 OCTOBER 2026

MultiSearch through the existing Milvus service

A native Python method and gRPC API that package independent searches across databases and collections into one call, with bounded dispatch through the existing Proxy Search path.

What this experiment demonstrates

The corrected implementation works over native gRPC on standalone Milvus. Under the two-CPU contention workload, MultiSearch achieved +8.2% logical throughput versus stock v3.0.5, with group p99 changing -5.3% and ordinary-Search probe p99 changing -7.1%. The modified ordinary Search path showed a +3.3% group-p99 increase and -0.4% throughput change, which remains a review caveat. Observed CPU utilization was 97.0% for stock and 97.0% for MultiSearch.

These are measurements of this workload, not a general latency or production-readiness guarantee. The Kubernetes evidence and remaining limits appear below.

---

1. The product contract

A consultation may need searches in multiple medical collections, each with its own vector dimension, metric, filter, output fields and topK. MultiSearch transports a list of those independent requests. Every item includes its own database and collection. Results stay separate and retain input order; there is no score fusion or cross-collection atomic snapshot.

```
client.multi_search([
    {"id": "papers", "db_name": "literature", "collection_name": "papers",
     "data": [paper_vector], "anns_field": "embedding", "limit": 10},
    {"id": "guidelines", "db_name": "clinical", "collection_name": "guidelines",
     "data": [guideline_vector], "anns_field": "vector", "limit": 3},
], max_concurrent_searches=8, timeout=5)
```

This extends the actual `MilvusClient`, `GrpcHandler` and `MilvusService`. The public RPC is `/milvus.proto.milvus.MilvusService/MultiSearch`. It uses the existing authenticated client channel. The protocol and SDK changes are provided as companion patches to their actual repositories; they are unpublished dependencies.

Transport batching reduces client-to-Proxy RPC count, not the underlying search work. Under an illustrative independent 3% per-RPC failure model, 100 calls have a 95.2% probability of at least one failure. One batched call has one transport exposure, but loses all its results if that call fails. Real PSC failures can be correlated; this prototype does not establish a 3% production question failure rate or validate any GCP SLA claim.

2. Why the wait does not consume a parent search worker



The concern about tying up threads is valid if the parent is put in the same finite worker pool as its children: all pool slots could wait for children that cannot run. Here, the parent stays outside the DQL pool. `WaitGroup.Wait()` and channel receives park Go goroutines instead of spinning or reserving an OS thread. The ordinary Search tasks still occupy their normal scheduler slots while executing. Waiting retains request/result state, and scheduling, serialization, allocation and leaf computation still cost CPU.

Current HybridSearch converts one database/collection request into a SearchRequest with SubReqs, then enqueues one search task and performs collection-local execution/reduction. It is a useful example of waiting for search completion, but its shared collection and ranking semantics do not provide independent cross-database dispatch. MultiSearch reuses the ordinary Search path separately for each item.

Existing task wait

```
func (tc *TaskCondition) WaitToFinish() error {
    select {
    case <-tc.ctx.Done():
        return errors.Wrap(tc.ctx.Err(), "proxy TaskCondition context Done")
    case err := <-tc.done:
        return err
    }
}
```

Actual dispatcher implementation

The code below is extracted from the prototype. The semaphore is shared by all batches on this Proxy. The worker count is bounded per batch. The parent waits outside the search scheduler.

```
parallelism := e.limits.Parallelism
if requested := req.GetMaxConcurrentSearches(); requested > 0 {
    parallelism = min(parallelism, int(requested))
}
// The RPC handler and these bounded workers stay outside the DQL pool.
for range min(parallelism, len(req.Items)) {
    wg.Add(1)
    go func() {
        defer wg.Done()
        for ctx.Err() == nil {
            i := int(next.Add(1) - 1)
            if i >= len(req.Items) {
                return
            }
            select {
            case e.active <- struct{}{}:
            case <-ctx.Done():
                return
            }
            if ctx.Err() != nil {
                <-e.active
                return
            }
            item := req.Items[i]
            // Search and entry interceptors may mutate the request. Never share
            // caller-owned request pointers between workers (or retry attempts).
            search := proto.Clone(item.Search).(*milvuspb.SearchRequest)
            result, err := e.search(ctx, search)
            <-e.active
            outcome := &milvuspb.SearchOutcome{Id: item.Id, Result: result}
            if err != nil {
                s := status.Convert(err)
                outcome.Result = nil
                outcome.GrpcCode, outcome.GrpcMessage = int32(s.Code()), s.Message()
            } else if result == nil || result.Status == nil {
                outcome.Result = &milvuspb.SearchResults{Status:
merr.Status(merr.WrapErrServiceInternalMsg("Search returned no result status"))}
            }
            // Reserve protobuf framing overhead as well as outcome bytes.
            if resultBytes.Add(int64(proto.Size(outcome)+16)) > int64(e.limits.MaxResponseBytes) {
                overflow.Store(true)
                cancel()
                return
            }
            out.Items[i] = outcome
        }
    }()
}
wg.Wait()
```

Actual integration with the ordinary Search entry

A Go method call does not automatically invoke gRPC interceptors. The callback explicitly re-enters the shared Search chain before calling the existing server method. Request cloning and independent mutable RPC statistics prevent sibling state sharing.

```
entry := proxyEntryInterceptor(limiter, getMetaCache)
search := func(ctx context.Context, req *milvuspb.SearchRequest) (*milvuspb.SearchResults, error) {
    // Fork mutable stats; the access-log interceptor similarly allocates
    // an independent object. Identity, deadlines and tracing are inherited.
    ctx = metrics.WithIndependentRPCStats(ctx)
    resp, err := entry(ctx, req, &grpc.UnaryServerInfo{Server: s, FullMethod:
milvuspb.MilvusService_Search_FullMethodName},
        func(ctx context.Context, req any) (any, error) {
            return s.Search(ctx, req.(*milvuspb.SearchRequest))
        })
    if err != nil {
        return nil, err
    }
    result, ok := resp.(*milvuspb.SearchResults)
    if !ok || result == nil {
        return &milvuspb.SearchResults{Status: merr.Status(merr.WrapErrServiceInternalMsg("Search entry returned
an invalid response"))}, nil
    }
    return result, nil
}
```

Actual public protocol additions

```
message MultiSearchRequest {
  repeated MultiSearchItem items = 1;
  // Zero uses the server default. Positive values are capped by server policy.
  uint32 max_concurrent_searches = 2;
}
message MultiSearchItem {
  string id = 1;
  SearchRequest search = 2;
}
message SearchOutcome {
  string id = 1;
  SearchResults result = 2;
  int32 grpc_code = 3;
  string grpc_message = 4;
}
message MultiSearchResponse {
  common.Status status = 1;
  repeated SearchOutcome items = 2;
}
```

Actual Python RPC call and item preparation

```
def multi_search(self, searches, max_concurrent_searches=0, timeout=None, context=None):
    """Execute independent searches with one RPC and no whole-batch retry."""
    if not searches:
        raise ParamError(message="multi_search requires at least one search")
    if not isinstance(max_concurrent_searches, int) or not 0 <= max_concurrent_searches < 2**32:
        raise ParamError(message="max_concurrent_searches must be a uint32")
    request = milvus_types.MultiSearchRequest(max_concurrent_searches=max_concurrent_searches)
    decimals = []
    for i, source in enumerate(searches):
        args = dict(source)
        item_id = str(args.pop("id", i))
        child_context = args.pop("context", context)
        db_name = args.pop("db_name", child_context.get_db_name() if child_context else "")
        child_context = CallContext(db_name=db_name)
        decimals.append(args.get("round_decimal", -1))
        child = self._prepare_search_request(timeout=timeout, context=child_context, **args)
        child.db_name = db_name
        request.items.add(id=item_id, search=child)
    response = self._stub.MultiSearch(request, timeout=timeout, metadata=_api_level_md(context))
    check_status(response.status)
```

---

3. Concurrency controls and established precedent

Elasticsearch's established _msearch API batches independent searches, including searches targeting different indices. Its `max_concurrent_searches` bounds concurrent sub-searches; the documented default is max(1, data nodes × min(search thread-pool size, 10)). Its `max_concurrent_shard_requests` separately limits each sub-search's concurrent shard requests per node, defaulting to 5. These distinguish request-level and shard-level pressure. This is a mature API pattern, not evidence that Milvus automatically inherits Elasticsearch's performance or fairness properties.

Prototype control	Default	Purpose
max_concurrent_searches (request)	0 = server default	Caller can lower item concurrency; cannot exceed the server cap.
proxy.multiSearch.parallelism	16	Maximum dispatcher workers per envelope.

globalParallelism	64 per Proxy	Shared active child Search calls across envelopes.
maxBatches / maxItems	8 / 128	Fail-fast envelope admission and bounded fan-out.
Request / response limits	4 / 16 MiB	Bound accepted payload and retained aggregate response.
timeoutSeconds	30	Upper bound on parent context; an earlier client deadline wins.

These controls are not a reservation for ordinary Search, a per-tenant fair scheduler or a cluster-wide concurrency limit. The prototype adds no new shard-concurrency control. The capacity tests use 128 shared permits and 32 admitted batches so MultiSearch is not compared with an artificially smaller logical-inflight limit.

---

4. Reproducible build and functional evidence

The stock reference is the latest stable 3.0 release found at test time: v3.0.5, source `af15c807513eb6508ae22d0f3a6484e8937be1a5`. This is a released-build comparison, not a claim about every later master commit. The modified Go binary uses the stock image's unchanged native libraries.


```
{
  "milvus_base_tag": "v3.0.5",
  "milvus_base_commit": "af15c807513eb6508ae22d0f3a6484e8937be1a5",
  "milvus_proto_base_commit": "2f00c7cf17db3ab3bb7b93ea8f2a7261bcc264da",
  "pymilvus_base_commit": "f63a1aa2816205c165f04d33e658b06e9b1cee11",
  "stock_image": "milvusdb/milvus@sha256:b9b79b4a284b41cd12d60abc418310f84621fa4c977d409b38f501519a7ccf8a",
  "native_libraries": "unchanged from the stock v3.0.5 image",
  "build_tags": "dynamic,sonic,with_jemalloc,bytedance_tango",
  "go": "go1.26.9",
  "pgo": "configs/pgo/default.pgo (empty in source checkout)",
  "server_cpu_cores": 4,
  "server_cpuset_active": "64-67",
  "server_cpuset_inactive": "68-71",
  "load_generator_cpuset": "32-39",
  "storage": "bounded 4 GiB tmpfs; CPU/read benchmark, not disk I/O",
  "rows": 160000,
  "collections": 8,
  "databases": 2,
  "index": "FLAT",
  "dimensions": [
    128,
    192,
    256
  ],
  "metric_types": [
    "L2",
    "IP",
    "COSINE"
  ],
  "modified_binary_sha256": "627c5efeb448deeb0d6e61151e463a756dbafad1e6d0b9d64762e2d28ef32a36",
  "stock_binary_sha256": "8ea7eaad6eab04babe04b0eab6bbec4be58153f0df720aa3825177ed9fff70ee",
  "cpu_model": "Intel Xeon Gold 6338N @ 2.20GHz",
  "host_logical_cpus": 128,
  "host_physical_cores": 64,
  "host_shared": true,
  "cpu_pinning_note": "CPU sets restrict scheduling; sibling hyperthreads and NUMA memory placement are not
reserved.",
  "saturation_cpu_limit": 2,
  "saturation_cpuset": "64-65",
  "saturation_GOMAXPROCS": 4
}
```


The standalone fixture contains 160,000 deterministic vectors across two databases, each with the same four collection names. Dimensions are 128/192/256; metrics are L2/IP/COSINE; indexes are FLAT. Fifty-item functional requests vary topK and compare IDs, fields and scores with ordinary Search.

Verification	Result
dispatcher tests	9
dispatcher race tests	passed
per item rpc stats race test	passed
proxy server and stream regression	passed
new sdk tests	3
existing sdk search tests	7
standalone native api tests	12
full repository suite run	False
kubernetes	passed: 12 native API checks on each of two Proxy replicas (24 checks)
performance	30 runs completed: 24 four-CPU runs and 6 two-CPU contention runs
sdk search tests after formatting	10

Coverage includes independent cross-database RBAC, missing databases, invalid vector dimensions, duplicate IDs, item and byte limits, deadlines, cancellation recovery, permit cleanup and status preservation. Race tests exercise dispatch and mutable statistics. The full repository test suite was not run. Dynamic quota rejection was traced through the shared interceptor but not fault-injected in this v3.0.5 run.

Distributed Kubernetes check

The same image is deployed in an isolated namespace on a single-node Kubernetes v1.30 development cluster: two Proxy replicas plus separate mixcoord, QueryNode, DataNode, StreamingNode, etcd and MinIO pods. A separate fixture has 2,048 vectors across the same two-database/eight-collection layout. Both Proxy replicas are addressed directly through local port forwarding, and the full native API verification suite is run against each. The running binary hash is recorded. This is a functional distributed-path check, not a multi-node availability or performance test.

---

5. Performance methodology

All paths execute identical groups of eight logical searches. Search sends eight concurrent ordinary RPCs; MultiSearch sends one RPC with eight items. Latency measures the time to complete the whole group in both cases. Performance items use nq=1 and topK 5–8; loaded batches contain eight items. Fifty-item batches are tested functionally, not benchmarked here. A compiled Go client avoids measuring Python fan-out limitations as server performance. Requests are pre-built and use the same vectors, parameters and authorization.

Each four-core capacity condition runs twice with reversed variant order, after five seconds of warmup and for 30 seconds of measurement. Logical concurrency caps are 8, 64 and 128. Offered-load runs at 480 and 1,200 logical searches/s are reported separately. Two-core saturation conditions run twice for 45 seconds each. CPU means are normalized to the actual quota and cover the entire standalone process, including Proxy, QueryNode and background

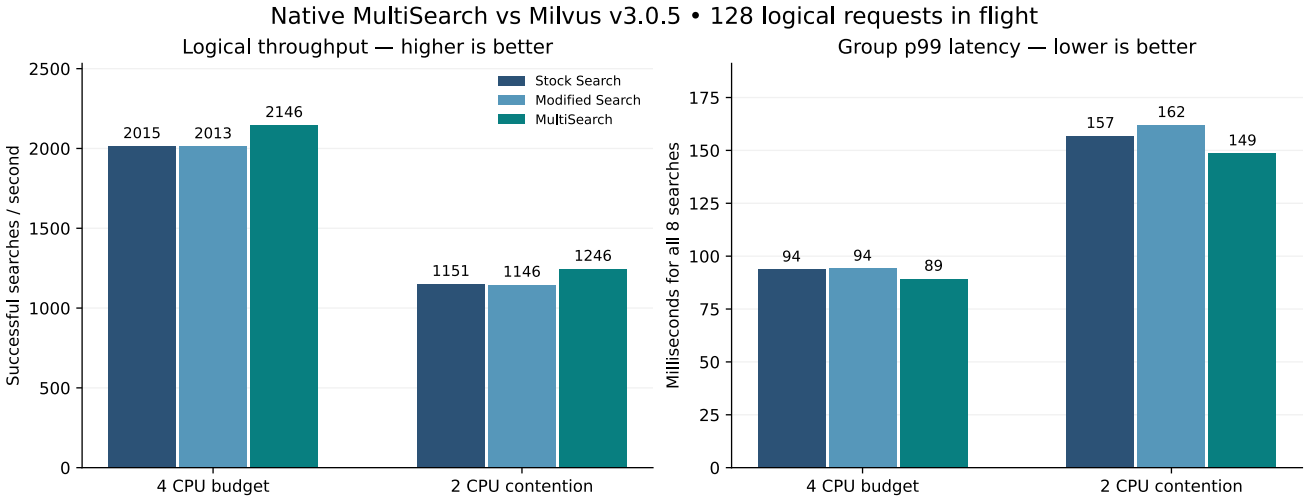
components; they do not isolate Proxy CPU cost. GOMAXPROCS remains 4 in both binaries during the two-core stress; this deliberately tests contention rather than a tuned two-core deployment.

The active standalone process is pinned to CPUs 64–67 (64–65 for saturation); the inactive instance remains alive on a separate cpuset. The client uses CPUs 32–39. Each container has a 16 GiB memory limit and bounded 4 GiB tmpfs data storage. The host is shared; CPU affinity does not reserve sibling hyperthreads or NUMA memory placement. This is an in-memory CPU/read benchmark, not a durable-storage, PSC or WAN experiment. Initial pause/resume runs were discarded after embedded-etcd leases expired.

An ordinary Search probe shares the server with each load. It attempts a request every 50 ms but waits for each response, so slow responses reduce its sampling rate. Its p99 is diagnostic, not a constant-arrival SLA measurement. Tables pool raw latency observations across repeats; throughput and CPU are means. No confidence intervals or statistical-significance claims are made.

---

6. Results at high CPU utilization



Path	Logical searches/s	CPU / limit	Group p99, ms	Search probe p99, ms	Errors
Stock v3.0.5 / Search	1,151.1	97.0%	156.84	132.27	0
Modified / Search	1,146.1	96.9%	161.96	128.91	0
Modified / MultiSearch	1,245.9	97.0%	148.58	122.82	0

Absolute degradation under contention: reducing the CPU budget from four cores to two changes MultiSearch group p99 from 89.32 to 148.58 ms (+66.4%), while throughput changes from 2,146.4 to 1,245.9 logical searches/s. Stock Search group p99 changes from 93.68 to 156.84 ms. The claim is about relative overhead at the same CPU budget, not immunity to CPU saturation.

At two cores, modified ordinary Search throughput changed -0.4% relative to stock, while its group p99 changed +3.3%. That small observed tail-latency regression needs more repeats and a matched rebuild control before attribution to the API integration; it is not dismissed as noise. MultiSearch changed throughput +8.2%, group p99 -5.3%, and the co-running ordinary-Search probe p99 -7.1%. Interpret both throughput and latency: more completed work is not sufficient evidence if it worsens unrelated requests.

Path	Throttled periods	Peak Go threads*	Peak goroutines	Peak cgroup memory, MiB	Probe samples
Stock v3.0.5 / Search	4.3%	45.0	1159	848	1174
Modified / Search	6.0%	44.0	1168	844	1162
Modified / MultiSearch	10.0%	44.0	970	841	1278

*Go runtime threads are not the total native thread count. Cgroup memory includes cache/tmpfs accounting; it is not Go heap size. Bounded peak goroutines in short runs do not prove absence of a long-duration leak.

Four-core capacity: 128 logical requests in flight

Path	Logical searches/s	CPU / limit	Group p99, ms	Search probe p99, ms	Errors
Stock v3.0.5 / Search	2,014.5	85.2%	93.68	75.08	0
Modified / Search	2,013.3	85.1%	94.09	73.70	0
Modified / MultiSearch	2,146.4	84.4%	89.32	75.88	0

Four-core capacity: 64 logical requests in flight

Path	Logical searches/s	CPU / limit	Group p99, ms	Search probe p99, ms	Errors
Stock v3.0.5 / Search	1,656.4	83.2%	61.98	51.55	0
Modified / Search	1,665.7	83.1%	60.83	49.33	0
Modified / MultiSearch	1,772.7	82.3%	56.14	47.10	0

Four-core capacity: 8 logical requests in flight

Path	Logical searches/s	CPU / limit	Group p99, ms	Search probe p99, ms	Errors
Stock v3.0.5 / Search	922.4	70.7%	18.15	16.24	0
Modified / Search	931.5	70.9%	18.18	13.67	0
Modified / MultiSearch	950.0	68.3%	17.36	13.04	0

Matched offered load (four cores)

Run	Offered logical/s	Completed logical/s	CPU	Group p99, ms	Probe p99, ms	Errors / dropped groups
multi-multi-rate1200	1200	1200.0	79.9 %	41.23	30.31	0 / 0
multi-multi-rate480	480	480.0	53.3 %	31.53	22.93	0 / 0
multi-search-rate1200	1200	1200.0	80.5 %	46.58	34.63	0 / 0
multi-search-rate480	480	480.0	54.7 %	31.64	24.96	0 / 0
stock-search-rate1200	1200	1200.0	80.9 %	52.11	41.67	0 / 0
stock-search-rate480	480	480.0	54.6 %	29.00	21.77	0 / 0

Across 30 recorded performance runs: 1,314,456 logical group searches, 0 group/probe errors, 0 dropped groups, 990 measurement seconds, excluding warmup. Raw samples and per-run resource telemetry accompany the report.

---

7. Failure semantics and production follow-ups

- Successful siblings survive individual item failures. Ordinary SearchResults and Status are preserved; transport errors retain code/message only. Full gRPC details and trailers are not retained.
- The SDK does not automatically retry the entire batch. A lost envelope response can repeat successful work if an application retries it; selective retry policy needs an explicit design.
- Envelope deadline/cancellation and aggregate response overflow fail the unary response. There is no partial-result streaming or resume token.
- Permits bound active Search calls, not all backend work remaining after cancellation. Transient allocations can exceed the retained-response budget.
- Large batches concentrate work on one Proxy. Tenant fairness, distributed balancing, larger batches, HNSW/other indexes, larger results, slower heterogeneous backends and long-duration saturation require additional testing. Kubernetes testing here is functional, not a cluster performance claim.
- Private Cloud gateway/billing hooks, envelope byte accounting, hot configuration updates and broader SDK data types need production review. REST and AsyncMilvusClient are not implemented.
- The protocol and PyMilvus patches need normal upstream releases and dependency pins. The build recipe uses released native libraries, not a full C++/Rust rebuild. The checked-in PGO profile is empty; exact release PGO provenance was not reconstructed.

8. Engineering assessment

The prototype demonstrates that independent multi-database searches can be added at the existing MilvusService/Proxy boundary without changing QueryNode execution or imposing common schemas and metrics. Bounded dispatch keeps the parent out of the DQL pool and preserves per-item authorization. The recorded CPU-contention measurements address the waiting/thread concern for this fixture; they do not eliminate the need for resource limits or establish fairness for arbitrary workloads.

Review `internal/proxy/multisearch.go`, `internal/distributed/proxy/multisearch.go`, the shared interceptor integration in `service.go`, and the two companion patches. Reproduction scripts, unit tests, functional tests and the load driver live under `examples/multisearch`. The design document is `docs/design-docs/multisearch.md`.

Evidence generated from local test artifacts. Latest-stable release reference checked on 2026-10-31. Prototype scope and unverified behavior are stated explicitly.